

Day 1: 技术回顾答疑讲解 + 项目Kickoff (5月24日)

技术选型

前端

React + Vite + React Router + Axios快速开发、组件化、单页路由、请求封装

后端

Java 21 + Spring Boot + Spring Web + Spring Data JPA构建 REST API，业务分层

数据库交互

数据库MySQL 存储航班、机场、订单等数据

构建

Maven & Spring Boot

项目构建管理

容器Docker前后端独立容器化

部署云平台

AWS ECR + ECS 镜像托管、服务部署

Day 1: 技术回顾答疑讲解 + 项目Kickoff（5月24日）

项目结构标准

```
flight-client/
├── src/
│   ├── pages/ # 页面级组件 (LoginPage, BookingPage)
│   │   ├── HomePage.jsx # 首页 (航班搜索)
│   │   ├── SearchResultPage.jsx # 航班列表页
│   │   ├── FlightDetailPage.jsx # 航班详情页
│   │   ├── BookingReviewPage.jsx # 预订确认页
│   │   ├── MyBookingsPage.jsx # 我的订单页
│   │   └── LoginPage.jsx # 登录页
│   ├── components/ # 通用组件 (FlightCard, FormField)
│   ├── services/ # axios 封装和 API 模块
│   │   ├── http.js # axios 实例配置
│   │   └── flightApi.js # 航班相关 API 请求函数
│   ├── context/ # 全局状态 (AuthContext, BookingContext)
│   ├── hooks/ # 自定义 hook (useAuth, useBooking)
│   ├── utils/ # 工具函数 (格式化、价格处理等)
│   ├── assets/ # 静态资源 (logo, 图标)
│   ├── App.jsx
│   └── main.jsx
├── public/
├── .env # REACT_APP_API_URL
└── Dockerfile
```

Day 1: 技术回顾答疑讲解 + 项目Kickoff（5月24日）

后端目录结构
(Spring Boot)

- flight-api/
 - controller/ # 控制器层: FlightController, AuthController, BookingController
 - service/ # 业务逻辑层
 - repository/ # 数据访问层 (JpaRepository)
 - entity/ # 实体类, 如 Flight, User, Booking
 - dto/ # 请求/响应 DTO, 如 LoginRequestDTO, BookingResponseDTO
 - config/ # JWT、CORS 配置类
 - exception/ # 全局异常处理, 如 GlobalExceptionHandler
 - util/ # 工具类 (时间格式化、Token解析)
 - resources/
 - application.yml
 - schema.sql # 数据库建表语句
 - data.sql # 初始化测试数据
 - Dockerfile
 - pom.xml

Day 1： 技术回顾答疑讲解 + 项目Kickoff（5月24日）

统一接口规范

	□ 接口路径	≡ 方法	≡ 请求参数	≡ 响应字段	≡ 状态码
	/api/flights	GET	from (string), to (string), date (yyyy-MM-dd)	List<FlightDTO> : id, flightNumber, departure, destination, date, time, price	200, 400
	/api/flights/{id}	GET	id (path variable)	FlightDetailDTO : 航班基本信息 + 舱位/航司图标等	200, 404
	/api/auth/login	POST	email, password	token, userInfo	200, 401
	/api/auth/register	POST	email, password, firstName, lastName, country, phone (optional)	token, userInfo	201, 400, 409
	/api/airports	GET	无	List<AirportDTO> : code, name, city	200
	/api/bookings	GET	Authorization Header, optional: status, page, size	List<BookingDTO> : reference, route, date, status	200, 401
	/api/bookings/{id}	GET	id (path variable), Authorization Header	BookingDetailDTO : 参考号、乘客、价格、航段等	200, 404, 401
	/api/bookings	POST	flightId, passengerInfo[]	bookingReference, totalPrice	201, 400, 401

Day 1: 技术回顾答疑讲解 + 项目Kickoff (5月24日)

开发规范

API 响应结构规范

```
{  
  "success": true,  
  "code": 200,  
  "message": "OK",  
  "data": { }  
}
```

```
{  
  "success": false,  
  "code": 400,  
  "message": "Invalid request",  
  "data": null  
}
```


Day 1: 技术回顾答疑讲解 + 项目Kickoff (5月24日)

命名规范与编码约定

前端命名规范

- 页面组件命名统一使用 Page 后缀 (如 FlightSearchPage.jsx)
- API 文件统一放入 services/, 按模块分 (如 flightApi.js, authApi.js)
- 状态管理使用 useXXXContext() + Context.Provider

后端命名规范

Controller 类统一使用 XxxController, 如 FlightController

DTO 分为:

FlightResponseDTO (返回数据)

FlightSearchRequestDTO (查询参数封装)

接口返回统一使用 BaseResponse<T>